HANDS-ON CODING ASSESSMENT / SET 03

Smart Energy Analytics

Analyze electricity meter readings, tariffs, substations and maintenance activity using PySpark in Google Colab. All files contain synthetic practice data.

Assessment datasets

CSV file	Rows	Primary purpose
meter_readings_90000.csv	90,000	Main meter readings and quality flags
meter_readings_extra_12000.csv	12,000	Union and duplicate detection
meter_master.csv	150	Site type, tariff and substation keys
substation_master.csv	12	City, region and capacity
tariff_master.csv	5	Tariff rates and categories
maintenance_tickets_1600.csv	1,600	Maintenance and ticket status

Data notes

The two readings files contain 102,000 rows in total, including 700 repeated reading_id values. Some readings intentionally contain null values in energy, voltage, power factor, temperature and operator fields.

Google Colab setup

```
!pip -q install pyspark
from google.colab import files
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.window import Window
from pyspark import StorageLevel
uploaded = files.upload()  # Browse and select all six CSVs
spark = SparkSession.builder.appName("EnergyAssessment").getOrCreate()
def load_csv(filename):
    return spark.read.csv(filename, header=True, inferSchema=True)
readings_df = load_csv("meter_readings_90000.csv")
```

Instructions: Upload all six CSVs, load each as a Spark DataFrame, and complete the tasks below. Use the DataFrame API unless a task explicitly asks for a different approach. Submit a runnable Colab notebook.

SECTION 01 / DATA PREPARATION & ANALYSIS

Tasks 01-13

A. Inspect, select and transform

01. Load all six CSVs. Display the schema, row count and first 10 rows of the main readings DataFrame.
02. Select reading_id, meter_id, reading_timestamp, energy_kwh and reading_status. In a separate DataFrame, rename source_type to collection_mode.
03. Filter readings where reading_status is Valid and energy_kwh exceeds 20. Sort by energy consumed, highest first.
04. Create energy_band using when() / otherwise(): Low below 5 kWh, Medium from 5 through 25 kWh, High above 25 kWh, and Unknown for nulls.

B. Null values and data cleaning

05. Calculate null counts for energy_kwh, voltage_v, power_factor and operator_id.
06. Create a new DataFrame that fills missing operator_id with UNASSIGNED and missing power_factor with 0.90. Do not modify the original DataFrame.
07. Remove readings with missing energy_kwh or voltage_v. Report the number of rows removed.

C. Joins, groupBy and aggregations

08. Join readings with meter_master on meter_id. Calculate total energy for Valid readings by site_type.
09. Extend the join to substation_master. Calculate total Valid energy by region and city.
10. Join tariff_master through meter_master. For non-Rejected readings with non-null energy, calculate energy_charge_inr = energy_kwh * rate_per_kwh_inr. Do not add the monthly fixed charge to every reading.
11. Join maintenance tickets with meter_master. Find ticket counts and average maintenance_cost_inr by site_type and severity.
12. Calculate average voltage_v and total outage_minutes by substation. Show the five substations with the highest total outage minutes.
13. Calculate the number of Valid, Estimated and Rejected readings for each source_type, ordered by reading count.

Tip: Keep intermediate DataFrames with meaningful variable names. Avoid using collect() on large, unaggregated datasets.

SECTION 02 / ADVANCED DATAFRAME OPERATIONS

Tasks 14-25

D. Date and window functions

14. Convert reading_timestamp into a Spark timestamp. Extract the year and month and calculate monthly Valid energy consumption by region.
15. Find the top three meters by total Valid energy within each substation using dense_rank().
16. Calculate cumulative Valid energy for each meter using a window ordered by reading timestamp and reading ID.

E. Union and deduplication

17. Combine both readings files using unionByName(). Compare row counts before and after the union.
18. Identify repeated reading_id values, remove duplicates using that key, and report the final unique reading count.

F. Lazy execution, caching and persistence

19. Build a pipeline with filter(), withColumn(), groupBy() and orderBy(). Inspect it using explain() before calling an action.
20. Cache Valid readings, run count() twice, and record the elapsed time for each action. Check is_cached.
21. Persist readings with outage_minutes > 0 using MEMORY_AND_DISK. Execute two actions and then release the data using unpersist().
22. Compare repeated execution of the same aggregation with and without caching. Record observations; do not assume caching must be faster.

G. Partitions and output

23. Repartition the main readings DataFrame into eight partitions and show the number of rows in each using spark_partition_id().
24. Start from 12 partitions. Reduce to three using coalesce(3) and separately using repartition(3). Compare the execution plans.
25. Join cleaned readings with meter and substation details, then save the enriched DataFrame as Parquet using four output partitions. Inspect the resulting part files.

SECTION 03 / INTEGRATED ASSESSMENT

Final Capstone

Energy Consumption & Reliability Report

Build an end-to-end PySpark pipeline that produces a monthly report for each region and site_type. Use the main and additional readings files and the relevant master datasets.

01 Combine the two readings datasets and deduplicate by reading_id.

02 Apply a documented cleaning rule for null energy values. Exclude Rejected readings from billable consumption.

03 Join meter, substation and tariff information using the appropriate keys.

04 Create a month column from reading_timestamp and calculate total kWh and estimated energy charges.

05 Calculate total outage minutes and the percentage of readings that report an outage for each region and site type.

06 Rank site types within each region by total billable energy charges.

07 Cache or persist the final aggregated report, run two actions, and release the cached data.

08 Write the report as Parquet with two output partitions.

Submission checklist

☐ One runnable Google Colab notebook with clearly labelled exercise numbers.

☐ PySpark code for all completed tasks; do not replace the work with Pandas.

☐ Visible outputs for key aggregations, rankings, and row-count checks.

☐ Execution-plan evidence for lazy execution and partitioning tasks.

☐ The final monthly report and its Parquet output directory.

Important assessment rules

Use the uploaded CSV files as the source data. Treat missing numeric values carefully; avoid silently treating missing readings as real consumption. When a task uses a tariff rate, apply the per-kWh rate only to relevant readings. The fixed monthly charge is not a per-reading charge.

Assessment focus: DataFrames, transformations, filtering, joins, aggregation, null handling, dates, windows, union, lazy execution, caching, persistence, repartition, coalesce and Parquet output.